LangChain Interview Notes

LangChain 1.x – concepts, code, and interview answers

Prepared for Dunsb

1 August 2026

Contents

LangChain Interview Notes — v1 (LangChain 1.x)

For: Dunsb · ~2 yrs AI engineering · targeting AI / Agentic AI Engineer roles **Written:** 1 August 2026 · verified against `docs.langchain.com` the same day **Target:** you can *answer* it and you can *code* it, from memory

---

How to use these notes

Every section has the same four parts:

Block	What it's for
Concept	The one-paragraph version. What it is, why it exists.
Code	Runnable, v1-correct. Type it, don't skim it.
Gotchas	Where it bites, and what interviewers probe.
Interview	Real questions + the answer shape that scores well.

Three rules for using this document:

1. **Reading these notes is not studying.** Code alongside them. A section you've only read is a section you'll fumble under pressure.
2. **Anything marked** [v1]** is a v1 change from what your old tutorial taught.** These are the highest-value corrections in the document.
3. **Don't spend more than a week here.** LangChain is already your strongest area. The market report rated it STRONG; your gaps are evaluation, cost, security, observability. This is a reference to *return to*, not a syllabus to grind. See the study plan at the end.

Version note: LangChain v1 shipped October 2025. It is a big break from the v0 material most tutorials teach. Everything below is v1.

Setup for everything below

```
pip install -U langchain langchain-openai langgraph \
               langgraph-checkpoint-sqlite langchain-text-splitters \
               python-dotenv
```

```
# .env
OPENAI_API_KEY=sk-...
```

Azure (your stack) — substitute anywhere a model is created:

```
from langchain_openai import AzureChatOpenAI

llm = AzureChatOpenAI(
    azure_deployment="your-deployment-name",
    api_version="2024-10-21",
    temperature=0,
)
```

Part 0 — The 60-second mental model

Concept

LangChain v1 is much smaller than v0. The `langchain` package now holds only:

Module	Contents
`langchain.agents`	`create_agent`, `AgentState`
`langchain.messages`	message types, `trim_messages`
`langchain.tools`	`@tool`, `BaseTool`
`langchain.chat_models`	`init_chat_model`
`langchain.embeddings`	`init_embeddings`

[v1] Everything else — legacy chains, `LLMChain`, old memory, old retrievers, the indexing API, the hub — moved to **`langchain-classic`**.

The single most useful sentence to hold in your head:

> **In v1, LangChain is largely LangGraph with a friendlier front door.**

`create_agent` builds a LangGraph graph. Memory is a LangGraph checkpointer. Human-in-the-loop is a LangGraph interrupt. This is *good news for you* — LangGraph is your strongest skill, so most of "learning LangChain v1" is applying what you already know.

The five lavers

```
┌──────────────────────────────────────────────┐
│  ORCHESTRATION   your control logic          │  <- you write this
├──────────────────────────────────────────────┤
│  AGENT           create_agent (LangGraph)    │  <- the loop
├──────────────────────────────────────────────┤
│  TOOL CALLING    bind_tools -> tool_calls    │  <- a REQUEST, not a call
├──────────────────────────────────────────────┤
│  LCEL            Runnables, the | operator   │  <- composition
├──────────────────────────────────────────────┤
│  MODEL           init_chat_model             │  <- the engine
└──────────────────────────────────────────────┘
```

Read Part 17 if the top two layers feel blurry — that confusion is normal and it's worth resolving properly.

Part 1 — Models

Concept

A chat model takes a list of messages and returns one `AIMessage`. `init_chat_model` builds one from a `"provider:name"` string, which makes model switching a config change instead of a code change.

Code

```
from langchain.chat_models import init_chat_model

llm = init_chat_model("openai:gpt-4o-mini", temperature=0)

# provider is inferred if unambiguous
llm = init_chat_model("gpt-4o-mini")

# swap providers by changing a string — this is "model switching"
llm = init_chat_model("anthropic:claude-sonnet-4-5")

# direct instantiation when you need provider-specific params
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0, max_tokens=1000, timeout=30)
```

Invoking:

```
llm.invoke("Hello")                                  # str shorthand
llm.invoke([HumanMessage("Hello")])                  # message list
llm.invoke([{"role": "user", "content": "Hello"}])   # dict shorthand

llm.batch(["a", "b", "c"])        # parallel
for c in llm.stream("Hello"):     # token by token
    print(c.text, end="")
await llm.ainvoke("Hello")        # async
```

Runtime config without rebuilding:

```
configurable = init_chat_model(
    "openai:gpt-4o-mini",
    configurable_fields=("model", "temperature"),
)

configurable.invoke("hi", config={"configurable": {"model": "gpt-4o"}})
```

Gotchas

- **`temperature=0` is not determinism.** You still get variation. This matters for eval design: assert on statistical thresholds over N runs, not exact string equality. Interviewers like this answer because it shows you've actually tested LLM systems.
- **Model switching is rarely free.** Prompts tuned for one model degrade on another; tool-calling reliability differs a lot between providers. The honest answer to "how do you switch models?" is *"changing the string is trivial — re-running the eval suite is the actual work."*
- **[v1]** `.text` is a **property** in v1, not a method. `.text()` still works but warns and is removed in v2.

Interview

Q: How would you support multiple LLM providers? `init_chat_model` with a provider string from config, so the model is injectable. Then the real work: a provider-agnostic eval suite, because prompt quality and tool-calling reliability differ per model. Mention fallbacks (Part 13) and per-request cost differences. Naming the eval suite is what separates this answer from a docs recital.

Q: How do you pick a model per request? Middleware with `@wrap_model_call` — cheap model for simple steps, strong model for hard ones. This is **model routing** and it's a top-three screening topic (cost). See Part 9.

Part 2 — Messages

Concept

Everything in LangChain is a list of messages. Understanding the four types and how they sequence *is* understanding agents.

Type	Who	Notes
`SystemMessage`	you	instructions; usually first, once
`HumanMessage`	user	input
`AIMessage`	model	has `.content` **and** `.tool_calls`
`ToolMessage`	your code	tool result; **must** carry `tool_call_id`

Code

```
from langchain.messages import (
    SystemMessage, HumanMessage, AIMessage, ToolMessage, trim_messages
)

messages = [
    SystemMessage("You are a helpful assistant."),
    HumanMessage("What is 2+2?"),
]
response = llm.invoke(messages)

response.content        # str or list of blocks
response.text           # always str  (PROPERTY, not method)
response.tool_calls     # list of {name, args, id}
response.usage_metadata # {'input_tokens': .., 'output_tokens': .., 'total_tokens': ..}
response.response_metadata
```

The sequence that defines an agent — memorise this:

```
HumanMessage("add 2 and 3, then times 4")
AIMessage(content="", tool_calls=[{name:"add", args:{a:2,b:3}, id:"c1"}])
ToolMessage(content="5", tool_call_id="c1")
AIMessage(content="", tool_calls=[{name:"multiply", args:{a:5,b:4}, id:"c2"}])
ToolMessage(content="20", tool_call_id="c2")
AIMessage(content="The answer is 20.")     # no tool_calls -> loop ends
```

Trimming to fit context:

```
trimmed = trim_messages(
    messages,
    max_tokens=4000,
    strategy="last",          # keep most recent
    token_counter=llm,
    include_system=True,      # never drop the system message
    start_on="human",         # valid sequences start on a human turn
)
```

Gotchas

- **Every tool call needs exactly one `ToolMessage` with a matching `tool_call_id`.** Miss one and the provider rejects the whole request. This is the #1 bug when hand-rolling loops.
- **When a model requests tools, `.content` is empty.** This is why `StrOutputParser` on a tool-calling chain gives you `""` — see Part 7.
- **[v1]** adds `.content_blocks` — a provider-agnostic normalised view (text / reasoning / image). Use it instead of writing per-provider branches on `.content`.

Interview

Q: Walk me through the message list for a two-tool agent run. Recite the six-line sequence above. Emphasise that `ToolMessage` is appended by *your* code, and that the loop terminates when

an `AIMessage` comes back with no `tool_calls`.

Q: Conversation exceeds the context window. Options? Three, with tradeoffs: `trim_messages` (fast, loses old facts); summarisation (keeps gist, **lossy** — something is always dropped); offload to retrieval and pull back on demand (most faithful, adds latency and complexity). Naming the lossiness is what makes this a senior answer.

Part 3 — Prompts

Concept

Templates that turn variables into messages. Less central in v1 than v0 — `create_agent` takes a plain `system_prompt` string — but still needed for chains and for slotting history into a prompt.

Code

```python
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_template("Summarise: {text}")

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a {role}."),
    ("human", "{question}"),
])

# !! HISTORY GOES HERE — the correct construct
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are helpful."),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{user_input}"),
])

prompt.invoke({"chat_history": [...], "user_input": "hi"})
```

Gotchas

- **[v1] `("messages", "{chat_history}")` is not valid and will raise.** `"messages"` is not a role. Use `MessagesPlaceholder`. This exact line appears in a lot of 2024 tutorials, including yours.
- Valid roles: `"system"`, `"human"`/`"user"`, `"ai"`/`"assistant"`, `"placeholder"`.
- Literal braces in a template must be doubled: `{{` and `}}`. Bites people writing JSON examples into prompts.
- **Prompts are code.** Version them in git, review them, test them. "Treats prompts as throwaway strings" is a stated interview red flag.

Interview

Q: How do you inject conversation history into a prompt? `MessagesPlaceholder`. Then add: in v1 you rarely hand-build this, because `create_agent` + a checkpointer manages history for you (Part 10).

Q: How do you manage prompts across environments? Versioned files or a prompt registry, not inline strings. Changes go through review and must pass the eval suite in CI. Ties directly to Gap #1 and #8 in your report.

Part 4 — LCEL & Runnables

Concept

Everything composable is a **Runnable** with the same interface: `invoke`, `batch`, `stream`, plus async twins. The `|` operator chains them. You get parallelism, streaming, and async for free.

Code

```
from langchain_core.runnables import (
    RunnableLambda, RunnableParallel, RunnablePassthrough, RunnableBranch,
)
from langchain_core.output_parsers import StrOutputParser

# 1. Sequential
chain = prompt | llm | StrOutputParser()
chain.invoke({"text": "..."})

# 2. Plain functions are auto-coerced inside a pipe
chain = llm | StrOutputParser() | (lambda s: s.upper())

# 3. Explicit wrap (needed at the START of a chain)
clean = RunnableLambda(lambda t: " ".join(t.strip().split()))

# 4. Parallel — both branches run concurrently
parallel = RunnableParallel(
    summary=prompt_a | llm | StrOutputParser(),
    keywords=prompt_b | llm | StrOutputParser(),
)
parallel.invoke({"text": "..."})   # {"summary": ..., "keywords": ...}

# 5. Passthrough — carry the input forward alongside new keys
enriched = RunnablePassthrough.assign(
    word_count=lambda x: len(x["text"].split())
)

# 6. Branch — if/elif/else
branch = RunnableBranch(
    (lambda x: x["score"] < 600, RunnableLambda(lambda _: "REJECTED")),
    (lambda x: x["score"] < 750, RunnableLambda(lambda _: "MANUAL REVIEW")),
    RunnableLambda(lambda _: "APPROVED"),     # default, required
)
```

A realistic composition — RAG in one expression:

```
rag = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
rag.invoke("What is the claims deadline?")
```

The dict literal is implicitly a `RunnableParallel` — retrieval and passthrough run concurrently.

Free superpowers on any chain:

```
chain.batch([{...}, {...}], config={"max_concurrency": 5})
for chunk in chain.stream({...}): ...
await chain.ainvoke({...})
chain.with_retry(stop_after_attempt=3)
chain.with_fallbacks([backup_chain])
chain.get_graph().print_ascii()      # visualise
```

Gotchas

- **Coercion needs a Runnable on the left.** `func | llm` fails; `RunnableLambda(func) | llm` works. Inside a chain that's already started, bare functions are fine.
- **`RunnableBranch` needs a default.** No match and no default raises.

- **`RunnableBranch` cannot loop.** Conditional yes, iteration no. That boundary is where LangGraph starts (Part 19).
- **Types must line up.** `StrOutputParser` emits `str`; the next step must accept `str`. Most LCEL bugs are shape mismatches — `.get_graph().print_ascii()` is the fast way to see it.

Interview

Q: What is LCEL and why use it? A composition standard where every piece implements the same Runnable interface. You compose with `|` and inherit batching, streaming, async, retries, and fallbacks without writing them. The real payoff is that `.stream()` works on an arbitrary chain for free.

Q: Run two independent LLM calls at once? `RunnableParallel`, or just a dict literal in a chain. For batching one chain over many inputs, `.batch(..., config={"max_concurrency": N})`.

Q: Difference between `RunnableBranch` and an agent? `RunnableBranch` — *you* write the condition; deterministic; no loop. Agent — the *model* decides; dynamic; loops. Different tools for "business rules" vs "open-ended reasoning."

Part 5 — Output parsing & structured output

Concept

Two distinct things, often confused:

- **Output parsing** — post-process the model's text (`StrOutputParser`, `JsonOutputParser`).
- **Structured output** — constrain the model to emit a schema in the first place (`with_structured_output`, `response_format`).

Structured output is strictly better when available. Parsing free text into JSON is a fallback.

Code

```
from pydantic import BaseModel, Field
from typing import Literal

class RiskAssessment(BaseModel):
    """Underwriting risk assessment."""
    score: float = Field(description="Risk score 0-100")
    decision: Literal["approve", "reject", "review"]
    reasons: list[str] = Field(description="Factors behind the decision")

structured_llm = llm.with_structured_output(RiskAssessment)
result = structured_llm.invoke("Applicant age 45, sum insured 500000.")

result.decision   # "review"  -> a typed object, not a string
```

Inside an agent — **[v1]** changed this:

```
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy, ProviderStrategy

# ProviderStrategy — provider-native. More reliable. Preferred when supported.
agent = create_agent(model=llm, tools=tools,
                     response_format=ProviderStrategy(RiskAssessment))

# ToolStrategy — artificial tool call. Works with any tool-calling model.
agent = create_agent(model=llm, tools=tools,
                     response_format=ToolStrategy(RiskAssessment))

# Bare schema: defaults to ProviderStrategy, falls back to ToolStrategy
agent = create_agent(model=llm, tools=tools, response_format=RiskAssessment)

result = agent.invoke({"messages": [HumanMessage("...")]})
result["structured_response"]     # the typed object
```

[v1] Prompted output was removed in v1 — passing a prompt string to coax JSON is gone. LangChain dropped it as unreliable. Knowing *why* it was removed is a better answer than knowing that it was.

Gotchas

- **Field descriptions are prompt engineering.** They're sent to the model. `Field(description=...)` measurably improves extraction quality — a cheap win people miss.
- **Structured output can still fail** — refusals, truncation on `max_tokens`, validation errors. Wrap in try/except; consider `.with_retry()`.
- **`Optional` beats hallucination.** If a field may be absent, type it `Optional[str] = None`. Forcing a required field invites the model to invent one.
- **[v1]** `create_agent` state schemas must be `TypedDict`. Pydantic models and dataclasses are no longer accepted for *state* (they're still fine for `response_format`).

Interview

Q: How do you get reliable JSON out of an LLM? `with_structured_output` with a Pydantic schema — provider-native constrained decoding when available, artificial tool calling otherwise.

Prompt-and-parse is the last resort; v1 removed prompted output as unreliable. Then handle refusals and truncation explicitly.

Q: `ToolStrategy` vs `ProviderStrategy`? Provider-native is more reliable, but only some providers support it. `ToolStrategy` works with any tool-calling model. v1 defaults to provider and falls back.

Q: Extraction returns wrong values sometimes. Debug it. Layered: is the field description clear? Is the input even in context (chunking/truncation)? Is the schema over-constrained so the model must invent? Is it a parse failure or a comprehension failure? Then — build an eval set and measure per-field accuracy rather than eyeballing. That last sentence is the one that scores.

Part 6 — Tools

Concept

A tool is a Python function the model can request. The `@tool` decorator turns a function into a `BaseTool` by extracting its **name**, **docstring**, and **type hints** into a JSON schema that gets sent to the model.

Code

```
from langchain.tools import tool
from pydantic import BaseModel, Field

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny, 28C in {city}"

get_weather.name          # "get_weather"
get_weather.description   # "Get the current weather for a city."
get_weather.args_schema.model_json_schema()   # what the model actually sees
```

Explicit schema when you need richer descriptions:

```
class SearchInput(BaseModel):
    query: str = Field(description="Search query. Be specific.")
    max_results: int = Field(default=5, ge=1, le=50)

@tool("search_docs", args_schema=SearchInput)
def search_docs(query: str, max_results: int = 5) -> str:
    """Search internal documentation. Use for product or policy questions."""
    return "..."
```

Async, and errors:

```
@tool
async def fetch_record(record_id: str) -> str:
    """Fetch a customer record by ID."""
    async with httpx.AsyncClient() as c:
        r = await c.get(f"https://api.internal/records/{record_id}", timeout=10)
        r.raise_for_status()
        return r.text
```

Returning both a model-facing summary and raw data:

```
@tool(response_format="content_and_artifact")
def query_db(sql: str) -> tuple[str, list[dict]]:
    """Run a read-only SQL query."""
    rows = run(sql)
    return f"Returned {len(rows)} rows.", rows   # summary to model, rows kept as artifact
```

Keeps 5,000 rows out of the context window while your app still has them.

Gotchas

- **The docstring is the model's only instruction manual.** Vague docstring → wrong tool selected. This is a real production failure mode, not a style preference. Say *when to use* the tool, not just what it does.
- **Too many tools degrades selection.** Past roughly 10–20, accuracy drops. Fix with dynamic tool filtering (Part 9), not more prompt pleading.
- **[SECURITY] Validate arguments before executing.** The model's args are untrusted input. Passing them straight into SQL, a shell, or a filesystem path is an injection vector — a stated interview red flag. Pydantic validation is your gate.
- **[SECURITY] Least privilege per tool.** Read tools open; write/delete tools scoped, confirmed, or human-approved. "The agent can do whatever the API key allows" is a **deal-breaker** answer.

Interview

Q: How do you design a good tool? Narrow single purpose; a docstring that says when to use it; typed args with `Field` descriptions and constraints; validation before execution; errors returned as messages rather than raised; least-privilege scope. Add: I keep the count low and filter dynamically, because selection accuracy falls as the toolset grows.

Q: Agent calls the wrong tool. Debug it. Look at the schema the model actually saw (`args_schema.model_json_schema()`) before touching the prompt. Usually the description is ambiguous or two tools overlap. Then: are there too many tools? Is it systematic or query-dependent? Build eval cases for tool selection specifically — it's measurable, and most people never measure it.

Q: Your agent needs to call external APIs. How do you build and secure that? *(rubric Q6, verbatim)* Schema design → validation → timeouts → retry with backoff → error-as-ToolMessage. Security: least privilege per tool, deny-by-default on writes, allowlisted parameters, never interpolate model output into SQL/shell. Mention MCP vs native function calling and when each fits.

Part 7 — Tool calling — the corrected core

> **This is the section your old tutorial got wrong. It is also the one most likely to come up.**

Concept

`bind_tools()` attaches tool schemas to a request. The model replies with an `AIMessage` whose `.tool_calls` is a **request**: *"please run `add(a=2,b=3)`."*

LangChain does not execute anything. Execution is your code's job — or `ToolNode`'s, or `create_agent`'s.

So the common framing "tool calling only makes one call" is wrong in an interesting way. **Tool calling makes zero calls. It only ever asks.**

Code — what `bind_tools` actually returns

```
llm_with_tools = llm.bind_tools([add, multiply])
ai_msg = llm_with_tools.invoke([HumanMessage("What is 2 + 3?")])

ai_msg.content      # ''   <- EMPTY when tools are requested
ai_msg.tool_calls   # [{'name': 'add', 'args': {'a': 2, 'b': 3}, 'id': 'call_abc'}]
```

[v1] This is why `prompt | llm.bind_tools(tools) | StrOutputParser()` returns an empty string. `StrOutputParser` reads `.content`, which is empty. Nothing ran. Your old DEMO A doesn't do what it claimed — run `01_tool_loop_from_scratch.py a` and watch.

Code — the loop, by hand

Type this once. It converts "I've read about function calling" into "I've implemented function calling," and the difference is audible.

```
from langchain.messages import HumanMessage, SystemMessage, ToolMessage

TOOLS = [add, multiply]
BY_NAME = {t.name: t for t in TOOLS}
llm_with_tools = llm.bind_tools(TOOLS)

messages = [
    SystemMessage("You are a careful assistant."),
    HumanMessage("First add 2 and 3, then multiply the result by 4."),
]

for step in range(6):                       # circuit breaker
    ai_msg = llm_with_tools.invoke(messages)
    messages.append(ai_msg)

    if not ai_msg.tool_calls:               # THE exit condition
        break

    for tc in ai_msg.tool_calls:
        try:
            result = BY_NAME[tc["name"]].invoke(tc["args"])
        except Exception as exc:
            result = f"ERROR: {exc}"        # feed back, don't crash
        messages.append(
            ToolMessage(content=str(result), tool_call_id=tc["id"])
        )
else:
    raise RuntimeError("max steps exceeded")

print(messages[-1].text)
```

Shortcut: `tool.invoke(tc)` (passing the whole tool-call dict) returns a ready `ToolMessage`. Use the explicit form while learning so the id-matching stays visible.

Parallel tool calls — where you were right

A model **can** emit several tool calls in one message:

```
ai_msg.tool_calls
# [{'name': 'get_weather', 'args': {'city': 'Delhi'},  'id': 'c1'},
#  {'name': 'get_weather', 'args': {'city': 'Mumbai'}, 'id': 'c2'}]
```

Genuinely useful — execute them concurrently. **But the arguments come from the model's own reasoning, not from executing anything.** If it emits `add(2,3)` and `multiply(5,4)` together, that `5` is mental arithmetic. Fine when calls are independent; unsafe when step 2 depends on step 1's real output (a DB write, an API response, a computation the model would fumble).

Sequencing on real outputs requires the loop. Nothing else.

Gotchas

- Every tool call needs exactly one matching `ToolMessage`. Missing id → provider error.
- **Always cap iterations.** Without a circuit breaker an agent can loop until your budget is gone.
- Feed tool errors back as `ToolMessage` content — models often recover by retrying with corrected args. Crashing wastes the run.

Interview

Q: Explain how function calling works under the hood. What does the model see vs. what does your code handle? The model sees JSON schemas derived from your tool signatures and docstrings. It returns a structured request in `AIMessage.tool_calls` — name, args, id. Your code validates and executes, then appends a `ToolMessage` with the result keyed by `tool_call_id`, and re-invokes. The model never touches your functions. *(The rubric flags inability to explain this as a red flag.)*

Q: Difference between tool calling and an agent? Tool calling is one round: the model requests. An agent is the loop — execute, feed the real result back, re-invoke, repeat until no tool calls come back. The feeding-back is the substance; the loop is just what wraps it.

Q: If I bind two tools, can the model chain them? It can emit both in one message — parallel tool calling. But the second call's arguments come from its own reasoning, not from running the first. True sequencing on real output needs the loop.

Q: Your agent loops forever. Prevent it. Max-iteration circuit breaker; per-task token budget with a hard stop; detect repeated identical tool calls; timeouts per tool; cost alerting. Ideally catch it in eval before production by asserting on trajectory length.

Part 8 — Agents

Concept

An agent runs tools in a loop until a stop condition. **[v1]** In v1 the one true entry point is `langchain.agents.create_agent`, which builds a **LangGraph graph** with `model` and `tools` nodes. It is Part 7's loop, productionised.

[v1] What changed

v0 (your tutorial)	v1
`create_openai_functions_agent`	**removed**
`AgentExecutor`	→ `langchain-classic`
`langgraph.prebuilt.create_react_agent`	superseded
`prompt=`	`system_prompt=`
`pre_model_hook` / `post_model_hook`	middleware `before_model` / `after_model`
`tools=ToolNode([...])`	`tools=[...]`
pre-bound `model.bind_tools(...)`	**not accepted** — pass `tools=`
streaming node `"agent"`	`"model"`

Code

```
from langchain.agents import create_agent

agent = create_agent(
    model=llm,                        # or "openai:gpt-4o-mini"
    tools=[add, multiply],
    system_prompt="You are a careful math assistant.",
)

result = agent.invoke({"messages": [HumanMessage("2+3, then times 4")]})
print(result["messages"][-1].text)
```

Everything together:

```
from langgraph.checkpoint.memory import MemorySaver

agent = create_agent(
    model=llm,
    tools=[search, risk_score],
    system_prompt="You are an underwriting assistant.",
    checkpointer=MemorySaver(),            # memory  (Part 10)
    response_format=RiskAssessment,        # structured output (Part 5)
    middleware=[...],                      # hooks (Part 9)
    context_schema=Context,                # static per-run context
    name="underwriting_agent",             # node name in multi-agent graphs
)
```

Streaming the loop:

```
for chunk in agent.stream({"messages": [HumanMessage("...")]}, stream_mode="values"):
    last = chunk["messages"][-1]
    if last.tool_calls:
        print("calling:", [tc["name"] for tc in last.tool_calls])
    elif last.content:
        print(last.text)
```

The ReAct loop

Reason → act → observe, repeated:

```
Human: find the most popular headphones and check stock
  reason: popularity is time-sensitive, I need search
```

```
  act:    search_products("wireless headphones")
  observe: top result WH-1000XM5
  reason: need to confirm availability
  act:    check_inventory("WH-1000XM5")
  observe: 10 units
  reason: I can answer now
AI: WH-1000XM5, 10 in stock
```

Gotchas

- **Empty `tools=[]`** gives a single LLM node with no tool-calling.
- **Agent names:** `snake_case`. Spaces and special characters are rejected by some providers.
- **Non-determinism is the cost.** Same input, different trajectory. Plan evals around distributions, not exact matches.
- **The default loop has no budget.** Add limits via middleware.

Interview

Q: What does `create_agent` give you over a hand-rolled loop? Checkpointing, streaming, middleware hooks, structured output, tool error handling, and it's a LangGraph graph so it composes into larger systems. Then the credibility line: *"but I've written the loop by hand — it's request, execute, append ToolMessage, re-invoke."*

Q: Design a multi-agent system for X. How do agents coordinate, and what happens when one fails? *(rubric Q7)* Decompose by clear responsibility, not "more agents." State the coordination model (supervisor vs handoff vs shared state). Failure: per-agent timeouts, circuit breakers, fallback paths, prevent cascades, cap delegation depth to stop infinite loops. Observability: one trace id across all agents. **Then say when single-agent is better** — "assumes multi-agent always beats single-agent" is a listed red flag.

Part 9 — Middleware

Concept

[v1] New in v1, and the answer to a lot of "how would you add X" questions. Middleware intercepts the agent loop at defined points. It replaces v0's pre/post-model hooks and is more composable — stack several, reuse across agents.

Hook	Fires	Use for
`before_model`	pre LLM call	trimming, summarisation, PII redaction, input guardrails
`after_model`	post LLM call	output guardrails, HITL, content filtering
`wrap_model_call`	around LLM call	model routing, dynamic tool filtering, retries
`wrap_tool_call`	around tool exec	error handling, sandboxing, approvals

Code

```
from langchain.agents.middleware import (
    SummarizationMiddleware, HumanInTheLoopMiddleware,
    wrap_model_call, wrap_tool_call, dynamic_prompt,
    ModelRequest, ModelResponse, AgentMiddleware, AgentState,
)
```

Cost control — model routing (your Gap #2, in ~10 lines):

```
cheap = init_chat_model("openai:gpt-4o-mini")
strong = init_chat_model("openai:gpt-4o")

@wrap_model_call
def route_by_complexity(request: ModelRequest, handler) -> ModelResponse:
    model = strong if len(request.state["messages"]) > 10 else cheap
    return handler(request.override(model=model))

agent = create_agent(model=cheap, tools=tools, middleware=[route_by_complexity])
```

Security — least-privilege tools by role:

```
@wrap_model_call
def tools_by_role(request: ModelRequest, handler) -> ModelResponse:
    role = request.runtime.context.user_role
    if role == "viewer":
        allowed = [t for t in request.tools if t.name.startswith("read_")]
        request = request.override(tools=allowed)
    elif role == "editor":
        allowed = [t for t in request.tools if t.name != "delete_data"]
        request = request.override(tools=allowed)
    return handler(request)
```

Tool error handling:

```
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_tool_errors(request, handler):
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"Tool error: check your input and retry. ({e})",
```

```
            tool_call_id=request.tool_call["id"],
        )
```

Human in the loop:

```
HumanInTheLoopMiddleware(
    interrupt_on={
        "send_email": {
            "description": "Review before sending",
            "allowed_decisions": ["approve", "reject"],
        }
    }
)
```

Budget enforcement — custom middleware with state:

```
from typing_extensions import NotRequired

class BudgetState(AgentState):
    calls: NotRequired[int]

class BudgetMiddleware(AgentMiddleware[BudgetState]):
    state_schema = BudgetState

    def __init__(self, max_calls: int = 10):
        self.max_calls = max_calls

    def before_model(self, state, runtime):
        if state.get("calls", 0) >= self.max_calls:
            return {"jump_to": "end"}          # hard stop
        return None

    def after_model(self, state, runtime):
        return {"calls": state.get("calls", 0) + 1}
```

Dynamic prompt:

```
@dynamic_prompt
def by_expertise(request: ModelRequest) -> str:
    role = request.runtime.context.get("user_role", "user")
    base = "You are a helpful assistant."
    return f"{base} Be technical." if role == "expert" else f"{base} Explain simply."
```

Gotchas

- **[v1]** Custom state must be a **`TypedDict`** extending `AgentState`. Pydantic/dataclasses rejected in v1.
- Middleware defining its own state is preferred over `state_schema=` on `create_agent` — keeps concerns scoped.
- Order matters when stacking; they nest like decorators.

Interview

Middleware is the concrete answer to at least four rubric questions — cost (routing), security (tool scoping), guardrails (`after_model`), and HITL. Naming it shows you're on v1.

Q: How do you add guardrails? Layered: `before_model` for input (PII redaction, injection screening), `wrap_model_call` for tool scoping by permission, `after_model` for output filtering, `wrap_tool_call` + HITL for dangerous actions. Emphasise defence in depth — if the prompt is compromised, permission boundaries still hold.

Part 10 — Memory

Concept

[v1] The single biggest v1 change, and the best reframe to memorise:

> **"How do you do memory in LangChain?" → "You don't. Memory is a LangGraph checkpointer."**

Need	v1 mechanism
Short-term (this conversation)	**checkpointer**, keyed by `thread_id`
Long-term (across sessions)	**store**
Context overflow	`SummarizationMiddleware` / `trim_messages`

All legacy memory classes were removed at 1.0 and live in `langchain-classic`.

Code

```
from langgraph.checkpoint.memory import MemorySaver

agent = create_agent(model=llm, tools=[], checkpointer=MemorySaver())

cfg = {"configurable": {"thread_id": "user-123"}}
agent.invoke({"messages": [HumanMessage("My name is Musaddik.")]}, config=cfg)
agent.invoke({"messages": [HumanMessage("What is my name?")]}, config=cfg)
# -> remembers
```

Note you pass **only the new message**. The checkpointer loads prior state and appends.

Durable:

```
from langgraph.checkpoint.sqlite import SqliteSaver          # single node
# from langgraph.checkpoint.postgres import PostgresSaver    # production

with SqliteSaver.from_conn_string("memory.sqlite") as cp:
    agent = create_agent(model=llm, tools=[], checkpointer=cp)
    agent.invoke({"messages": [...]}, config=cfg)
```

Inspect / resume without invoking:

```
state = agent.get_state(cfg)
state.values["messages"]     # full history
state.next                   # what would run next -> crash recovery
```

Long-term memory across threads:

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
agent = create_agent(model=llm, tools=tools, checkpointer=MemorySaver(), store=store)
```

Context overflow:

```
SummarizationMiddleware(model=llm, trigger={"tokens": 4000})
```

[SECURITY] Thread isolation is NOT multi-tenancy

Different `thread_id` = different conversation. **But if your API takes `thread_id` from the client and passes it through, user A can read user B's history by guessing an id.** That's an IDOR vulnerability, not a feature.

Real multi-tenancy:

```python
def thread_for(session) -> str:
    return f"{session.tenant_id}:{session.conversation_id}"   # derived server-side

def handle(session, text):
    assert_owns(session.user_id, session.conversation_id)     # authorise FIRST
    cfg = {"configurable": {"thread_id": thread_for(session)}}
    return agent.invoke({"messages": [HumanMessage(text)]}, config=cfg)
```

Then **write a test proving cross-tenant reads fail.** That test is a portfolio artifact (Gap #9).

Gotchas

- No `thread_id` with a checkpointer configured → error. It's required.
- `MemorySaver` is process-local: dies on restart, and doesn't work across multiple workers. Use Sqlite/Postgres for anything real.
- Summarisation is **lossy**. Always. Know what yours drops.
- Checkpointer stores the whole graph state — including tool calls and results — not just a text transcript. That's why an agent can refer back to a tool result from three turns ago.

Interview

Q: How does memory work in LangChain? "In v1 memory is a LangGraph checkpointer keyed by `thread_id`. Short-term is the checkpointer, long-term is the store, overflow is summarisation middleware. The old memory classes were removed at 1.0 and moved to `langchain-classic`." — This one answer proves you're current.

Q: Agent crashes mid-task. How does it resume? Durable checkpointer (Postgres in prod). On restart, `get_state(config)` returns the last checkpoint including `state.next`. Re-invoke on the same `thread_id` and it continues rather than restarting.

Q: Keep tenant A's data out of tenant B's? Namespace the thread id server-side from the authenticated session, authorise before invoking, isolate the retrieval corpus per tenant too, and test that cross-tenant access fails. Explicitly note that `thread_id` alone is separation, not authorisation.

Part 11 — Retrieval / RAG

Concept

RAG fixes two LLM limits: finite context and frozen knowledge. Pipeline:

```
Sources -> Loaders -> Documents -> Split -> Embed -> Vector store
                                                        |
Query -> Embed ------------------------------------> Retrieve -> LLM -> Answer
```

[v1] In v1 the legacy `langchain.retrievers` module moved to `langchain-classic`. Loaders, splitters, embeddings, and vector stores live in integration packages (`langchain-openai`, `langchain-qdrant`, `langchain-text-splitters`, ...).

The three architectures — know when to use each

Architecture	Control	Latency	Use when
2-step RAG	High	Fast, predictable	FAQ, docs bot — retrieval is always needed
Agentic RAG	Low	Variable	Research assistant — model decides *whether* and *how* to search
Hybrid	Medium	Variable	Ambiguous queries needing validation / re-query

Agentic RAG is just "give an agent a retrieval tool." That's the whole trick.

Code — 2-step RAG

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,          # overlap prevents facts splitting across boundaries
)
chunks = splitter.split_documents(docs)

vectorstore = SomeVectorStore.from_documents(chunks, OpenAIEmbeddings())
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

prompt = ChatPromptTemplate.from_template(
    "Answer using ONLY the context. If the context is insufficient, say so.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

rag = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)
rag.invoke("What is the claims deadline?")
```

Code — Agentic RAG

```python
@tool
def search_policies(query: str) -> str:
    """Search internal insurance policy documents. Use for coverage or claims questions."""
    return format_docs(retriever.invoke(query))

agent = create_agent(
    model=llm,
    tools=[search_policies],
    system_prompt="Use search_policies for policy questions. Quote relevant snippets.",
)
```

Hybrid search + reranking — the depth that gets probed

Sarvam AI's posting names these explicitly: *"chunking, embeddings, hybrid search, reranking, vector databases."* If your production work has been dense-vector only, **this is where a sharp interviewer finds your edge.**

- **Dense** (embeddings) — semantic similarity. Fails on exact matches, rare terms, product codes, names.
- **Sparse** (BM25) — lexical. Catches the exact-match cases dense misses.
- **Hybrid** — run both, fuse. **Reciprocal Rank Fusion:** `score(d) = Σ 1/(k + rank_i(d))`, typically `k=60`. Rank-based, so you don't have to normalise incompatible score scales.
- **Reranking** — cross-encoder over the top ~50 → top 5. Bi-encoders embed query and doc separately (fast, indexable); cross-encoders read both together (slower, much more accurate). Retrieve wide, rerank narrow.

```python
def rrf(rankings: list[list[str]], k: int = 60) -> list[tuple[str, float]]:
    scores: dict[str, float] = {}
    for ranking in rankings:
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores.items(), key=lambda kv: -kv[1])
```

[DEBUG] Debugging RAG — rubric Q8, verbatim

> *"Your RAG pipeline retrieves relevant documents but the agent still gives wrong answers. Debug this."*

Debug **in layers**, independently:

1. **Retrieval** — is the right chunk in the top-k at all? Measure recall@k. If it's absent, it's a retrieval problem; stop here.
2. **Chunk boundaries** — is the fact *split across* two chunks, truncated, or sitting at an edge? Extremely common, rarely checked. Fix with overlap or semantic chunking.
3. **Context assembly** — how many chunks are you stuffing? Too many **dilutes** and the model ignores the key passage. Sweep top-k and plot — quality peaks then *declines*.
4. **Generation faithfulness** — does the answer actually follow from the retrieved context, or is the model going off-script from parametric knowledge? Measure faithfulness directly.
5. **Query-dependent or systemic?** Reformulate the query. If it fixes it, you have a query-understanding problem (→ query rewriting), not a retrieval one.

NO: Red flags: *"just increase top-k"* · can't tell retrieval failure from generation failure · doesn't know their own chunking strategy · no faithfulness metric.

Metrics to name

Metric	Measures
recall@k / precision@k	did retrieval surface it
MRR / NDCG	ranking quality
context precision / recall	RAGAS, retrieval-side
faithfulness	is the answer grounded in the context
answer relevancy	does it address the question

Interview

Q: Chunk size — how do you choose? Empirically, against an eval set — there's no universal number. Small chunks: precise retrieval, but facts fragment. Large: more context per hit, more noise and dilution. Overlap (~10–20%) mitigates boundary splits. Then: I measure recall@k and faithfulness across a few settings rather than guessing.

Q: When would you NOT use RAG? Stable knowledge that fits in context (just put it in the prompt, and cache the prefix). Queries needing computation or aggregation rather than lookup (use SQL/a tool). Genuinely needing new *behaviour* rather than new *facts* (fine-tuning). "RAG for everything" is as much a smell as "agents for everything."

Part 12 — Streaming

Concept

Every Runnable streams. For agents you choose *what* to stream.

Code

```python
for chunk in chain.stream({"text": "..."}):
    print(chunk, end="", flush=True)

# agents: stream_mode controls granularity
for chunk in agent.stream(payload, stream_mode="values"):   # full state each step
    ...
for chunk in agent.stream(payload, stream_mode="updates"):  # only the delta
    ...
for token, meta in agent.stream(payload, stream_mode="messages"):  # token-level
    print(token.text, end="")

async for event in agent.astream_events(payload, version="v2"):
    if event["event"] == "on_chat_model_stream":
        print(event["data"]["chunk"].text, end="")
```

Gotchas

- **Structured output doesn't stream usefully** — you need the whole object to validate it.
- **Tool calls stream as fragments.** Accumulate `AIMessageChunk`s before acting; don't execute on a partial call.
- **[v1]** the agent's streaming node is `"model"`, not `"agent"`.
- **[v1]** `AIMessageChunk` has `chunk_position` — `'last'` marks the final chunk.

Interview

Q: How do you stream an agent's output to a UI? `stream_mode="messages"` for tokens; `"updates"` for step-level progress ("calling search_policies..."). In practice both: progress events for tool steps, tokens for the final answer. Mention that tool-call chunks must be accumulated, not acted on mid-stream.

Part 13 — Resilience

Concept

Two failure classes: **transient** (rate limits, timeouts, 5xx) → retry. **Persistent** (provider down, model deprecated) → fall back.

Code

```
resilient = llm.with_retry(
    retry_if_exception_type=(RateLimitError, APITimeoutError),
    wait_exponential_jitter=True,
    stop_after_attempt=3,
)

primary = init_chat_model("openai:gpt-4o")
backup  = init_chat_model("anthropic:claude-sonnet-4-5")
robust  = primary.with_fallbacks([backup])

# fallbacks compose over whole chains, not just models
chain = (prompt | primary | parser).with_fallbacks([prompt | backup | parser])
```

Tool errors → `wrap_tool_call` middleware (Part 9). Hand-rolled loops → try/except → `ToolMessage` (Part 7).

Gotchas

- **Don't retry non-transient errors.** Retrying a 400 burns money and latency for a guaranteed failure.
- **Always jitter.** Synchronised retries across workers create thundering herds.
- **Timeouts before retries.** No timeout means a hung request never reaches your retry logic.
- **Fallback models have different behaviour.** A prompt tuned for one may degrade on the other — your fallback path needs eval coverage too, or you've built a silent quality cliff.

Interview

Q: Provider has an outage mid-request. What happens? Timeout fires → retry with exponential backoff and jitter for transient classes only → fall back to a second provider → if all fail, degrade gracefully with a clear user-facing error rather than a stack trace. Add: the fallback path is in the eval suite, because a fallback nobody tested is a quality cliff you'll discover in production.

Part 14 — Async & parallelism

Concept

LLM calls are IO-bound. Async is where throughput comes from — and you'll need it for parallel eval runs (Gap #1).

Code

```python
result = await chain.ainvoke({...})
results = await chain.abatch([{...}, {...}], config={"max_concurrency": 10})
async for chunk in chain.astream({...}): ...

# concurrent, different inputs
import asyncio
results = await asyncio.gather(*(chain.ainvoke(x) for x in inputs))

# bound concurrency to respect rate limits
sem = asyncio.Semaphore(5)
async def bounded(x):
    async with sem:
        return await chain.ainvoke(x)
results = await asyncio.gather(*(bounded(x) for x in inputs))

# execute parallel tool calls concurrently
results = await asyncio.gather(*(
    BY_NAME[tc["name"]].ainvoke(tc["args"]) for tc in ai_msg.tool_calls
))
```

Gotchas

- Define tools with `async def` if they do IO. A sync tool in an async agent blocks the loop.
- Rate limits are the real ceiling — use `max_concurrency` or a semaphore.
- `.batch()` on a sync chain uses a thread pool; `.abatch()` is true async. Prefer async for IO.

Interview

Q: Process 10,000 documents through an LLM chain. How? `abatch` with bounded concurrency tuned to the rate limit; provider batch APIs (roughly 50% cheaper) if latency is not critical; checkpoint progress so a crash doesn't restart from zero; dead-letter queue for permanent failures; and estimate the total cost *before* starting. That last point is Gap #2 showing up in a design answer.

Part 15 — Observability

Concept

When a multi-step agent gives a bad answer, you need the whole trajectory. Named by Sarvam AI as an ownership area (Signoz, Datadog); LangSmith appears in 14.8% of LangGraph postings.

Code

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=ls__...
export LANGSMITH_PROJECT=my-agent
```

Tracing is automatic once set. Then:

```
from langsmith import traceable

@traceable(name="risk_pipeline")
def assess(applicant: dict) -> dict:
    ...

# correlate a trace with your own request id
agent.invoke(payload, config={
    "run_name": "underwriting",
    "tags": ["prod", "v2"],
    "metadata": {"user_id": "123", "request_id": req_id},
})
```

Langfuse is the open-source, self-hostable alternative — worth knowing by name.

What to log

Log	Don't log
prompt + completion (redacted)	raw PII
token counts in/out	secrets, API keys
latency per step	full documents (sample instead)
tool name, args, result, duration	
model + version, cost per call	
trace id across all agents	

Gotchas

- **[SECURITY] Redact PII before traces leave your process.** Traces are a data-exfiltration path people forget.
- **One trace id across the whole request** — otherwise multi-agent debugging is guesswork.
- Sampling in high volume: trace all errors, sample successes.

Interview

Q: A user says the agent answered wrong three hours ago. Find out what happened. Look up the trace by request id → read the trajectory: what was retrieved, which tools ran with what args, what each returned, what the model saw at each step. Isolate the failing layer (retrieval vs tool vs generation). Then **add that case to the eval set so it's caught next time.** The last sentence converts a debugging answer into a maturity answer.

Q: What do you alert on? Error rate, p95 latency, cost per request and daily spend, tool failure rate, average iterations per task (a spike means looping), and eval-score regression after a deploy.

Part 16 — MCP

Concept

Your differentiator — lead with it. Model Context Protocol is an open standard for exposing tools to models. Anthropic donated it to the Linux Foundation's Agentic AI Foundation in December 2025 (Google, Microsoft, AWS backing; 97M+ monthly SDK downloads).

Why it matters in the data: **MCP appears in 17–24% of postings for every one of the five major frameworks.** It is the *only* framework-agnostic companion in the market data. Sarvam AI's posting requires *"production MCP server experience"* — a phrase almost no 2-year candidate can honestly claim.

Code

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

client = MultiServerMCPClient({
    "filesystem": {
        "command": "npx",
        "args": ["-y", "@modelcontextprotocol/server-filesystem", "/data"],
        "transport": "stdio",
    },
    "internal_api": {
        "url": "https://tools.internal/mcp",
        "transport": "streamable_http",
    },
})

tools = await client.get_tools()          # MCP tools -> LangChain tools
agent = create_agent(model=llm, tools=tools)
```

Stateless by default — each call opens a session. For stateful sequences:

```
async with client.session("filesystem") as session:
    ...
```

MCP tool errors come back as a tool message with `status="error"` rather than raising, so the model can read and retry.

MCP vs native function calling

	Native `@tool`	MCP
Coupling	in your codebase	separate process/service
Reuse	per app	across apps, teams, clients
Versioning	your deploy	independent
Best for	app-specific logic	shared capability surface

Answer the "when would you use which" question with the coupling argument: native tools for logic that belongs to this app; MCP when a capability should be reusable across apps and independently deployable.

Interview

Q: What is MCP and why does it matter? An open protocol standardising how models discover and call tools — client/server, transport-agnostic (stdio, HTTP). It matters because it decouples tool implementation from the agent framework: the same server works from LangChain, Claude Desktop, Cursor. Linux Foundation governance since Dec 2025. In hiring data it's the only framework-agnostic skill.

Q: You've run MCP servers in production — what broke? *Prepare a real answer here.* Talk about transport choice, session lifecycle and statelessness, auth to downstream systems, timeout

and error propagation, versioning tool schemas without breaking clients. **This is the question where your actual experience separates you from everyone else in the pipeline. Do not waste it on generalities.**

Part 17 — Orchestration — the corrected hierarchy

> Your notes said orchestration is "the boss" of tool calling, agents, and workflows. **That's the right instinct with one wrong rung.** Here's the accurate version.

The corrected picture

```
ORCHESTRATION   <- YOUR control logic. Not a LangChain class.
    |
    +-- WORKFLOW   fixed sequence, YOU decide the order  (LCEL)
    +-- AGENT      dynamic loop, MODEL decides           (create_agent)
            |
            +-- TOOL CALLING   the model REQUESTS a call  (bind_tools)
                    |
                    +-- TOOL   your Python function
```

What your notes got right: orchestration is a strategy, not a class. It's your own control logic deciding which path runs. The cooking analogy holds.

What was wrong: tool calling was placed as a *sibling* of agents. It isn't — **it's a component inside them.** An agent is a loop *around* tool calling. Tool calling alone never executes anything (Part 7).

Layer	Who decides	Loops?	Executes tools?
Tool	—	—	it *is* the execution
Tool calling	model picks	**NO:**	**NO: only requests**
Agent	model	**YES:**	**YES:** (its loop does)
Workflow	you	**NO:** (branch only)	if you code it
Orchestration	you	your choice	delegates

Code — all four in one system

```python
def classify(text: str) -> str:
    t = text.lower()
    if "sum insured" in t or "age" in t:
        return "underwriting"
    if any(op in t for op in "+-*/"):
        return "math"
    return "general"

# WORKFLOW: fixed sequence, deterministic
underwriting_workflow = (
    RunnableLambda(clean_text)
    | RunnableLambda(lambda t: agent.invoke({"messages": [HumanMessage(t)]})["messages"][-1].text)
    | format_prompt | llm | StrOutputParser()
)

# AGENT: dynamic
general_agent = create_agent(model=llm, tools=TOOLS, system_prompt="...")

# ORCHESTRATION: your routing logic
def orchestrate(user_input: str) -> str:
    route = classify(user_input)
    if route == "underwriting":
        return underwriting_workflow.invoke(user_input)
    if route == "math":
        return calc_chain.invoke({"input": user_input})
    return general_agent.invoke({"messages": [HumanMessage(user_input)]})["messages"][-1].text
```

Interview

Q: Difference between a chain, an agent, and a workflow? Chain — fixed sequence of Runnables, you define the order, deterministic. Agent — the model decides which tools to call and when, loops until done, non-deterministic. Workflow — a chain with branching and business

rules; still developer-controlled. Orchestration is the layer above choosing between them. And: tool calling isn't a peer of these — it's the mechanism agents are built on.

Q: When do you use each? Deterministic business process with compliance requirements → workflow, every time; you need reproducibility and an audit trail. Open-ended user input where you can't enumerate the paths → agent. Most real systems are workflows with agents embedded at the genuinely uncertain steps.

Part 18 — When NOT to use an agent

> **Rubric Q3, and one of the three five-minute screening questions. A confident "here's when I chose *not* to" is a strong differentiator.**

The decision rule

Agents earn their place when the task needs **dynamic reasoning, tool selection, or adaptation to highly variable input**. Deterministic logic stays deterministic.

Situation	Use
Fixed steps, known order	chain / workflow
Business rules, compliance, audit trail	workflow — you need reproducibility
One tool, obvious when to call it	direct tool call
Structured extraction	`with_structured_output`
Query needs computation over data	SQL / a function
Genuinely open-ended, unpredictable paths	agent

The costs you must be able to name

- **Cost** — many LLM calls per task, and the whole message list is re-sent each iteration.
- **Latency** — sequential model calls, unpredictable count.
- **Non-determinism** — same input, different path. Hard to test, hard to support.
- **Debugging** — you're debugging a trajectory, not a stack trace.
- **Security surface** — tool access plus untrusted input.

The rubric cites a real case: a company going from $0/month to $47K/month in orchestration costs.

Interview

Q: When would you build an agent vs. just write regular code? *(rubric Q3, verbatim)* Give the decision rule, then **name a specific case where you chose not to** — that's the green flag. Something like: *"We had a document-processing flow with fixed validation rules. An agent could have done it, but the steps never varied, we needed an audit trail, and it would have been slower, costlier, and non-deterministic. We used a workflow and called the LLM only for the extraction step."*

NO: Red flags: everything should be an agent · can't name the downsides · has never decided against one.

Part 19 — LangChain limits → LangGraph

Concept

[v1] In v1 this framing is softer than your notes suggest — `create_agent` *is* LangGraph. The real question is **when do you drop to the graph API directly.**

What LCEL can't do

Need	LCEL	LangGraph
Sequential steps	**YES:**	**YES:**
Branching	**YES:** `RunnableBranch`	**YES:** conditional edges
Loops / cycles	**NO:**	**YES:**
Shared mutable state	**NO:** (data flows forward)	**YES:** typed state
Persistence / resume	**NO:**	**YES:** checkpointers
Human in the loop	**NO:**	**YES:** interrupts
Multi-agent	**NO:**	**YES:** subgraphs
Time travel / replay	**NO:**	**YES:**

The one-line boundary: *LCEL is a DAG — data flows forward and never comes back. The moment you need a cycle, durable state, or a pause for human input, you need a graph.*

Code — dropping to the graph API

```python
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from typing_extensions import TypedDict

class State(TypedDict):
    query: str
    docs: list
    answer: str
    attempts: int

def retrieve(s: State) -> dict:
    return {"docs": retriever.invoke(s["query"]), "attempts": s.get("attempts", 0) + 1}

def grade(s: State) -> str:                # conditional edge -> enables the CYCLE
    if not s["docs"] and s["attempts"] < 3:
        return "rewrite"
    return "generate"

g = StateGraph(State)
g.add_node("retrieve", retrieve)
g.add_node("rewrite", rewrite_query)
g.add_node("generate", generate_answer)
g.add_edge(START, "retrieve")
g.add_conditional_edges("retrieve", grade, {"rewrite": "rewrite", "generate": "generate"})
g.add_edge("rewrite", "retrieve")          # <-- the cycle LCEL cannot express
g.add_edge("generate", END)

app = g.compile(checkpointer=MemorySaver())
```

Interview

Q: When do you reach for LangGraph over LCEL? When I need a cycle, durable state, human-in-the-loop, or multi-agent coordination. LCEL is a DAG. Self-correcting RAG — retrieve, grade, rewrite, retry — is the canonical example: it needs a loop with a counter, and there's no way to express that in a pipe chain.

Q: How do you prevent infinite loops in a graph? Iteration counter in state checked by the conditional edge; LangGraph's `recursion_limit`; a token/cost budget with a hard stop; and detecting repeated identical states.

Part 20 — Question bank

Answer each **out loud** before reading the answer. Recognising an answer is not the same as producing one, and the gap between them is exactly what an interview measures.

Difficulty: *(warm-up)* warm-up · *(core)* core · *(senior signal)* senior signal

A. Fundamentals

***(warm-up)* 1. What is LangChain and what problem does it solve?** A framework for building LLM applications: standard interfaces over models, tools, and retrieval so you can compose and swap components. v1 narrowed it sharply to agent building blocks; legacy chains moved to `langchain-classic`. The honest framing: it's most valuable for the standard interface and the agent runtime, less so as a grab-bag of chains.

***(warm-up)* 2. What is LCEL?** A composition standard where every component is a Runnable with `invoke`/`batch`/`stream` plus async twins, chained with `|`. You inherit batching, streaming, async, retries, and fallbacks without implementing them.

***(core)* 3. Walk me through the message types.** `SystemMessage` (instructions), `HumanMessage` (input), `AIMessage` (has `.content` *and* `.tool_calls`), `ToolMessage` (result, must carry `tool_call_id`). Then recite the agent sequence from Part 2 — it demonstrates you understand the loop, not just the vocabulary.

***(core)* 4. `.batch()` vs `.abatch()`?** `batch` uses a thread pool over the sync path; `abatch` is true async. For IO-bound LLM calls prefer async. Both take `max_concurrency`.

***(senior signal)* 5. `temperature=0` — is output deterministic?** No. Batching, hardware nondeterminism, and provider-side changes all introduce variation. Consequence: eval assertions must be statistical (N runs, threshold) rather than exact-match, and "it worked when I tried it" is not evidence.

B. Tools & tool calling

***(core)* 6. What does `bind_tools` actually do?** Serialises tool schemas into the provider's format and attaches them. The model replies with `AIMessage.tool_calls` — a *request*. **It executes nothing.**

***(senior signal)* 7. Explain function calling under the hood — model vs your code.** See Part 7. The cleanest possible answer: *"The model sees JSON schemas and returns a structured request. My code validates, executes, appends a `ToolMessage` keyed by `tool_call_id`, and re-invokes. The model never touches my functions."*

***(core)* 8. Tool calling vs agent?** One round vs a loop. The loop's substance is feeding real tool output back as a `ToolMessage` and re-invoking.

***(core)* 9. Bind two tools — can it chain them?** It can emit both in one message (parallel tool calling), but arguments come from its own reasoning, not from executing the first. True sequencing needs the loop.

***(senior signal)* 10. Agent picks the wrong tool. Debug.** Inspect the schema the model saw first, before touching the prompt. Overlapping or vague descriptions are the usual cause. Check tool count. Determine systematic vs query-dependent. Build tool-selection eval cases and measure.

***(senior signal)* 11. Securing tool access?** *(rubric Q6)* Least privilege per tool; deny-by-default on writes; Pydantic validation between the model's args and execution; allowlisted parameters; never interpolate model output into SQL/shell/paths; timeouts and retries; errors as messages. Layered so a compromised prompt still can't reach dangerous tools.

C. Agents & memory

(core) **12. How does `create_agent` work?** Builds a LangGraph graph with `model` and `tools` nodes. Model node emits tool calls → tools node executes and appends `ToolMessages` → back to model → until no tool calls or a limit trips.

(core) **13. Memory in LangChain?** "You don't — memory is a LangGraph checkpointer keyed by `thread_id`. Short-term is the checkpointer, long-term is the store, overflow is summarisation middleware. Legacy memory classes were removed at 1.0."

(senior signal) **14. Design memory for 6-month user relationships.** Two tiers. Short-term: checkpointer per conversation thread. Long-term: extract durable facts and store them, retrieve **by relevance, not recency**. Compaction policy with an explicit view on what's lossy. Per-tenant isolation. Then: I'd measure what summarisation drops rather than assume it's fine.

(senior signal) **15. Agent crashes mid-task. Resume?** Durable checkpointer (Postgres). `get_state(config)` returns the last checkpoint including `state.next`. Re-invoke on the same thread id. Caveat: side effects already executed aren't rolled back — so tools that mutate need idempotency keys.

(senior signal) **16. Multi-tenancy?** Namespace `thread_id` server-side from the authenticated session; authorise before invoking; isolate the retrieval corpus per tenant; test that cross-tenant reads fail. Say explicitly: `thread_id` alone is separation, **not** authorisation — client-supplied ids are an IDOR bug.

D. RAG

(core) **17. Walk me through a RAG pipeline.** → Part 11 diagram, plus where each piece fails.

(senior signal) **18. Retrieves relevant docs but answers wrong. Debug.** *(rubric Q8)* → The five-layer method in Part 11. Lead with "I'd separate retrieval quality from generation faithfulness and test each independently."

(core) **19. Dense vs sparse vs hybrid?** → Part 11. Include RRF and *why* rank-based fusion avoids score normalisation.

(core) **20. What does a reranker add?** Cross-encoder reads query and document together instead of embedding them separately — much better precision, too slow to run over the whole corpus. So: retrieve wide with a bi-encoder, rerank narrow. Quote both the quality gain and the latency cost.

(senior signal) **21. Recall@k is 95% but answers are still wrong.** It's not retrieval. Look at context assembly (dilution from too many chunks), chunk boundaries (fact split/truncated), prompt (is the model told to ground strictly?), and faithfulness (is it answering from parametric knowledge instead?).

(core) **22. Chunk size?** Empirically against an eval set. Explain the tradeoff and overlap.

E. Production

(senior signal) **23. Manage cost across hundreds of LLM calls?** *(rubric Q5 — five-minute screen)* Model routing (cheap for classification/routing, strong for reasoning), prompt caching on stable prefixes, per-task token budgets with hard stops, max-iteration caps, batch API where latency allows, spend monitoring and alerts. **Quote a real number for a system you built** — "cannot estimate what a system costs" is a red flag.

(senior signal) **24. How do you know when your agent is wrong?** *(rubric Q2)* Golden dataset built from **real production traffic**, not only hand-written cases. Separate scorers for retrieval, final answer, and trajectory. LLM-as-judge with an explicit rubric ("0 wrong, 1 vague, 2 correct and concise"), validated against human labels. Regression gate in CI. Say the human-agreement number if you have one — almost nobody does.

(senior signal) **25. Test an agent before deploying?** *(rubric Q4)* Golden dataset, automated scoring, baseline comparison, trajectory eval not just final answers, statistical thresholds for non-determinism, eval in CI blocking merge on regression, prompts versioned as reviewed artifacts.

(senior signal) **26. Defend against prompt injection?** *(rubric Q9)* **Lead with the distinction most candidates miss:** direct (user tries to override instructions) vs **indirect** (malicious instructions embedded in a retrieved document). Defences: treat all retrieved content as untrusted data; input/output separation; allowlisted tool parameters; layered permissions so a compromised prompt still can't reach dangerous tools; canary tokens; red-teaming before launch. **NO:** "We tell the model to ignore bad instructions" is not a security model.

(core) **27. Observability?** → Part 15. Traces, spans over tool calls, PII redaction, one trace id across agents.

(core) **28. Provider outage?** → Part 13. Timeouts → jittered retries on transient classes → fallback provider → graceful degradation.

F. Judgment

(senior signal) **29. Agent vs regular code?** *(rubric Q3)* → Part 18. **Name a real case where you said no.**

(senior signal) **30. Design a multi-agent system. One agent fails — then what?** *(rubric Q7)* → Part 8. Include "when single-agent is better."

(senior signal) **31. LangChain vs LangGraph?** → Part 19. "LCEL is a DAG; the moment you need a cycle, durable state, or a human pause, you need a graph." Note that in v1 `create_agent` already *is* a graph.

(senior signal) **32. Downsides of LangChain?** Real answer, delivered evenly: abstraction depth makes debugging harder than raw SDK calls; the API has churned significantly (v1 is a large break); some abstractions hide token/cost details you need visibility into; for a single simple call the provider SDK is often clearer. It's most valuable for the standard interface, the agent runtime, and the ecosystem — less so as a wrapper around one API call. **Being able to critique your main tool reads as senior, not disloyal.**

Part 21 — Coding drills

Time-boxed. **Write them without looking at the notes**, then diff against the relevant section. Getting stuck *is* the useful signal — it shows you which section you've only read.

Drill 1 — Tool loop from scratch · 30 min · [KEY] most important

No `create_agent`, no `AgentExecutor`. `bind_tools` + a `while` loop. Must have: multi-step sequencing on real outputs · max-iteration breaker · tool errors fed back as `ToolMessage` · token counting. **YES:** Done when: "add 2 and 3, then multiply by 4" produces two *sequential* tool executions with the real 5 flowing into the multiply.

Drill 2 — Parallel tool calls · 20 min

Extend Drill 1: when `tool_calls` has more than one entry, execute them concurrently with `asyncio.gather`. Compare wall-clock against sequential.

Drill 3 — Structured extraction · 20 min

Pydantic schema with 5+ fields including an enum and an optional. Extract from messy text. Add `Field(description=...)` everywhere and measure whether accuracy improves — **actually measure it.**

Drill 4 — Memory with threads · 20 min

`create_agent` + `MemorySaver`. Prove two `thread_id`s stay isolated. Swap to `SqliteSaver` and prove persistence across a restart. Then write the failing-by-design multi-tenancy test from Part 10.

Drill 5 — Router · 30 min

Classify input → route to (a) a direct chain, (b) a fixed workflow, (c) an agent. Log the chosen route. This is Part 17 in code.

Drill 6 — Hybrid retrieval · 45 min

Dense + BM25 over a small corpus, fused with RRF (write the function). Measure recall@5 for dense alone vs hybrid. Add a cross-encoder reranker and measure again — record the latency cost too.

Drill 7 — Cost middleware · 30 min

`@wrap_model_call` routing cheap↔strong by complexity. Track cost per request. Report the saving and any quality delta.

Drill 8 — Guardrail middleware · 30 min

`before_model` redacting emails/phone numbers from input; `after_model` blocking a disallowed pattern in output. Write 10 injection attempts — 5 direct, 5 indirect (embedded in a retrieved doc) — and record which get through.

Drill 9 — Self-correcting RAG graph · 60 min

`StateGraph`: retrieve → grade → (rewrite → retrieve)* → generate. Attempt counter, max 3. This is the canonical "why LangGraph" artifact.

Drill 10 — Streaming endpoint · 30 min

FastAPI + `StreamingResponse` over `agent.astream`. Emit tool-step progress events and answer tokens separately. Uses your existing FastAPI strength.

Appendix A — v1 migration cheat sheet

If you say any of the left column in an interview, you sound like you learned from 2024 tutorials.

NO: v0	**YES:** v1
`LLMChain(llm, prompt)`	`prompt | llm | parser`
`ConversationChain`	`create_agent(..., checkpointer=...)`
`ConversationBufferMemory`	`checkpointer=MemorySaver()`
`ConversationBufferWindowMemory`	`trim_messages` / `SummarizationMiddleware`
`ConversationEntityMemory`	long-term `store`
`VectorStoreRetrieverMemory`	`store` with semantic search
`create_openai_functions_agent`	`langchain.agents.create_agent`
`initialize_agent`	`create_agent`
`AgentExecutor`	`create_agent` (LangGraph graph)
`langgraph.prebuilt.create_react_agent`	`langchain.agents.create_agent`
`("messages", "{chat_history}")` **raises**	`MessagesPlaceholder(variable_name=...)`
`prompt=` on agents	`system_prompt=`
`pre_model_hook` / `post_model_hook`	middleware `before_model` / `after_model`
`tools=ToolNode([...])`	`tools=[...]`
`handle_tool_errors=` on ToolNode	`@wrap_tool_call` middleware
`response_format=Schema` (tool only)	`ToolStrategy` / `ProviderStrategy`
prompted structured output	**removed — was unreliable**
pre-bound `model.bind_tools(...)` into agent	pass `tools=`
Pydantic/dataclass agent state	**`TypedDict` only**
`.text()`	`.text` (property)
`config={"configurable": {...}}` for static ctx	`context=` + `context_schema=`
streaming node `"agent"`	`"model"`
`from langchain.retrievers import ...`	`langchain_classic.retrievers`
`from langchain import hub`	`from langchain_classic import hub`

Also: **Python 3.10+ required.** 3.9 support dropped.

Appendix B — 7-day plan

Hard cap: 7 days. Then go to evaluation, which is your actual bottleneck. The rubric treats framework unfamiliarity as a *minor* concern and no-eval-strategy as a **dealbreaker**. Every extra day here has sharply diminishing returns.

Day	Read	Build	Outcome
1	Parts 0–4	Drill 1 (**the big one**)	You can explain function calling under the hood
2	Parts 5–6	Drills 2, 3	Structured output + parallel tools
3	Parts 7–8	Rewrite an old project on `create_agent`	v0 habits gone
4	Parts 9–10	Drills 4, 7	Memory + cost middleware (Gap #2 starts here)
5	Part 11	Drill 6	Hybrid + rerank — upgrades your strongest area
6	Parts 12–16	Drills 8, 10	Guardrails, streaming, and your MCP story written down
7	Parts 17–19	Drill 9 + Part 20 out loud	Judgment answers; LangGraph boundary clear

Then stop

Move to **evaluation** (Gap #1) and **cost** (Gap #2). Those are 2 of 9 rubric questions and 1 of 3 stated dealbreakers between them. More LangChain will not get you the offer; an eval harness will.

Three things to have written down before any interview

1. **Your MCP story.** You have production MCP experience and almost no other candidate does. Rehearse 90 seconds on what you built, what broke, and what you'd change.
2. **Three production war stories.** Rubric Q1's green flag is *"talks about what went wrong."* What broke, how you found it, what you changed, what you'd do differently. This is the highest-value hour in the whole preparation and nearly nobody does it.
3. **One case where you chose NOT to use an agent.** Rubric Q3's green flag. Specific system, specific reasoning.

Sources

All API details verified against the live docs on **1 August 2026**:

- LangChain v1 migration guide
- Agents
- Retrieval
- `bind_tools` reference
- Tool Calling with LangChain (blog)
- langchain-mcp-adapters

Interview rubric material (green/red flags, scoring levels, the nine questions) is from the Agentic AI Engineer Hiring Guide — a practitioner guide written for hiring managers, cited throughout as a *rubric*, not as evidence from job postings.

[v1] LangChain moves fast. Re-check the migration guide before relying on these APIs months from now.